

Indian AI API startups are billing wrong

They charge in rupees for millions of micro-transactions per day - per 1,000 LLM tokens, per embedding, per inference. Four failure modes cost them real money:

- **Retries double-bill.** A network hiccup makes the gateway re-POST a usage event. Upstream keys its event store on the client timestamp, so a retry with a changed timestamp persists twice and the customer pays twice.
- **Mid-month upgrades mis-prorate.** Day-15 plan changes either fail outright or produce credits and charges that don't net to the right number.
- **Sub-paise rates round to zero.** INR precision is hardcoded to 2 decimals. At Rs 0.002 per token the charge rounds to 0 and the revenue disappears.
- **Invoices explain nothing.** A number with no breakdown of tokens, tiers, or proration is a support ticket waiting to happen.

What it costs them

2x billed per retried event a re-POST with a changed timestamp persists as a second row	24h dedup cache TTL upstream a batch replayed after it bypasses deduplication entirely	0 paise billed at Rs 0.002/token sub-paise rates round away before they reach the invoice	11 verified upstream gaps catalogued with file:line evidence in docs/GAPS.md
--	---	--	---

A deterministic billing engine, built from docs/ alone

Clean-room rebuild in Python 3 + FastAPI + SQLite. Zero external infrastructure: no Postgres, no ClickHouse, no Redis, no Kafka, no Temporal.



- **Money is never a float.** Amounts are integer paise (1 INR = 100). Unit rates are integer micro-paise (1 paise = 10,000). Half-up rounding happens exactly once, at the invoice line.
- **Time is injectable.** Nothing calls `datetime.now()`. `BILLING_NOW` from `.env` drives the clock, so a 30-day cycle and a Day-15 upgrade are simulated in milliseconds - the whole suite runs in ~1 second.
- **Durability before acknowledgement.** HTTP 201 is returned only after the row is committed. No fire-and-forget broker to lose it.

Three tests, written before a line of engine code

Every expected value below is quoted from docs/PRD.md section 5. Tests were committed red, then the engine was built until they passed.

KILLER TEST 1 PASS	KILLER TEST 2 PASS	KILLER TEST 3 PASS
Duplicate usage event counts once 50,000 raw tokens / divide_by 1,000 = 50 billable units Retry with a CHANGED timestamp -> HTTP 200 duplicate_skipped 8 concurrent threads on evt_race_99 -> exactly 1 row written Invoice line quantity = 50, never 100	Mid-month upgrade prorates exactly 30-day April cycle, upgrade on Day 15 (Apr 16 00:00 UTC) coefficient = 15 / 30 = 0.5000 Unused Starter credit = -150,000 paise (-Rs 1,500.00) Remaining Pro charge = +450,000 paise (+Rs 4,500.00) NET AMOUNT DUE = 300,000 paise (Rs 3,000.00)	Tiered pricing matches the hand calculation 1,500,000 tokens = 1,500 units against a 2-tier table SLAB: 1,000 x Rs 0.10 + Rs 5.00 = 10,500 paise + 500 x Rs 0.05 + Rs 10.00 = 3,500 paise = 14,000 paise = Rs 140.00 VOLUME: 1,500 x Rs 0.05 + Rs 10.00 = 8,500 paise = Rs 85.00

pytest: 35 passed - 16 Killer Test cases, 7 Fix, 12 Differentiator

One fix, one differentiator - both specified in docs/GAPS.md

FIX

Deterministic dedup + sub-paise rating

docs/GAPS.md Improvement 1 - closes gaps 1, 7, 11

Dedup at the storage layer

UNIQUE(tenant_id, event_id) in SQLite. A retry with a mutated timestamp, or a batch replayed 7 days later, still dedupes - there is no cache TTL to outlive.

Rate in micro-paise, round once

Rs 0.002/token = 2,000 micro-paise. Ten single-token requests bill 2 paise, not 0.

The defect the tests caught

Billable units were rounded through divide_by BEFORE any money was computed. 1,500,500 tokens billed 14,005 paise instead of the true 14,002.5 -> 14,003. Rating now stays in raw-token space over an exact integer numerator.

Acceptance criterion

Concurrent evt_retry_10 -> 1 row, 50,000 tokens, exactly 500 paise (Rs 5.00). Zero duplicate leakage.

DIFFERENTIATOR

Invoice explainer + runaway-spike alert

docs/GAPS.md Improvement 2 - closes gaps 8, 10

GET /v1/invoices/{id}/explanation

One paragraph explaining how raw tokens became billable units, which tiers applied, and the exact proration credit and charge.

GET /v1/customers/{id}/spike-status

Trailing-hour velocity against the 7-day hourly average. Over SPIKE_THRESHOLD_FACTOR (3.0x) it flags and logs - catching the agent stuck in a prompt loop.

AI is optional and cannot break the app

Key from .env. Missing key, missing package, bad key, API error, or a refusal all return HTTP 200 from the deterministic template, with source and ai_error on the response so the fallback is visible, not silent.

The model never does arithmetic

Every figure is computed locally from the sealed invoice; claude-opus-5-5 only rewrites those facts. An AI outage changes the wording, never the money.

Everything runs from two commands

```
python3 -m venv venv && ./venv/bin/pip install -r requirements.txt
cp .env.example .env
```

```
./venv/bin/python -m pytest -q      # 35 passed in ~1s
./scripts/demo                     # 67 live checks, all PASS
```

scripts/demo boots a real uvicorn server on a throwaway SQLite file and drives it over HTTP, printing every request with the EXPECTED value quoted from docs/PRD.md beside the ACTUAL response.

35**pytest cases passing**

16 Killer / 7 Fix / 12 Differentiator

67**live demo checks**

every one PASS

~1s**full suite runtime**

BILLING_NOW, no sleeps

0**external services**

SQLite only

Verified live in the demo

- + Duplicate and 8-way concurrent events count exactly once
- + Slab 14,000 paise / Volume 8,500 paise, to the paise
- + Invoice explanation returns 200 with no AI key configured
- + Day-15 upgrade nets exactly 300,000 paise (Rs 3,000.00)
- + Rs 0.002-per-token usage bills 2 paise instead of rounding to 0
- + Runaway loop flagged at 10.00x the 7-day baseline